

ECE 315

Steven Knudsen, PhD, PEng

Lecture 8 :

- More on RTOS scheduling
- Tasks and Scheduling
- Protecting Critical Code



UNIVERSITY
OF ALBERTA

Last Lecture

- Bare Metal Programming
- Strategies to ensure deterministic, low-latency performance

This Lecture

- More on RTOS
- Tasks and Scheduling
- Protecting Critical Code

Learning Objectives

- Define and understand context switching from a task perspective
- Describe a task control block and what it's used for
- Describe a pre-emptive, round-robin scheduler program flow
 - Answer under what conditions will a task not be pre-empted?
- Describe cooperative multi-tasking and its pros and cons
- Understand what is meant by a critical section of code
- Know ways to protect critical sections and their pros and cons
- Define Mutex and Semaphore and know the differences

Back to RTOS...



UNIVERSITY
OF ALBERTA

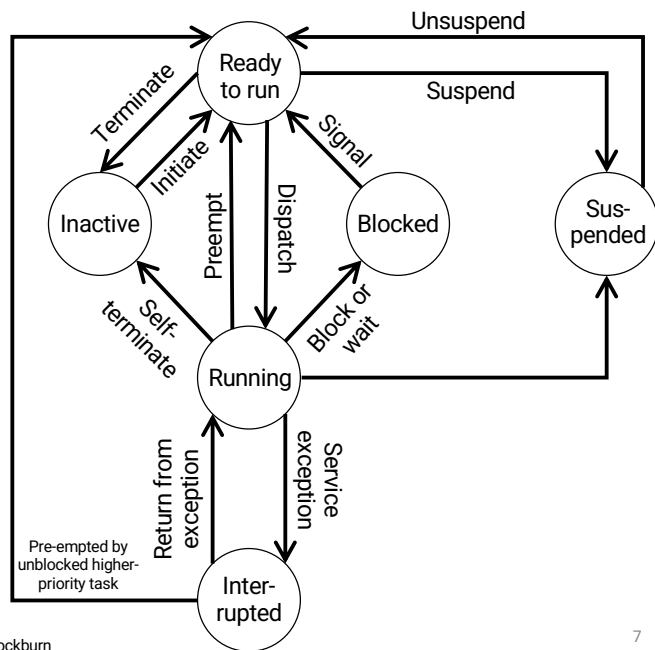
5

Tasks Re-visited - States

- **Running**: The one process that is actively executing instructions on the CPU (unless interrupted).
- **Interrupted**: The one running process has been stopped temporarily to allow an interrupt service routine to run.
- **Ready-to-run**: The process is available to start executing instructions as soon as the CPU becomes available.
- **Blocked**: The process is waiting for a message to arrive, or is waiting for a flag or semaphore to become free.
- **Suspended**: The process either suspended itself, or was suspended by another process. A longer-term block.
- **Inactive**: The process has been defined, but has either never been initiated or was terminated later.

Process States in a Multitasking Environment

- Only one task is running or interrupted at any one time
- Queues are used to record the tasks that are ready-to-run, inactive, blocked, and suspended.



UNIVERSITY
OF ALBERTA

Adapted from ECE 315 W2025 by Dr. Bruce Cockburn

Task Context and Persistence

- A **task** is an executing program together with the current context
- **context** comprises several components
 - the contents of all CPU registers
 - the contents of the task's stack in RAM
 - the contents of any other allocated RAM locations
 - the state of any other allocated system resources
- A task can be stopped at one time and restarted later if the task context is saved in a special data structure, which is often called a **task control block** (TCB).
- That is, the task is persisted across pre-emptions via the TCB

Task Control Block (TCB)

A **TCB** may contain

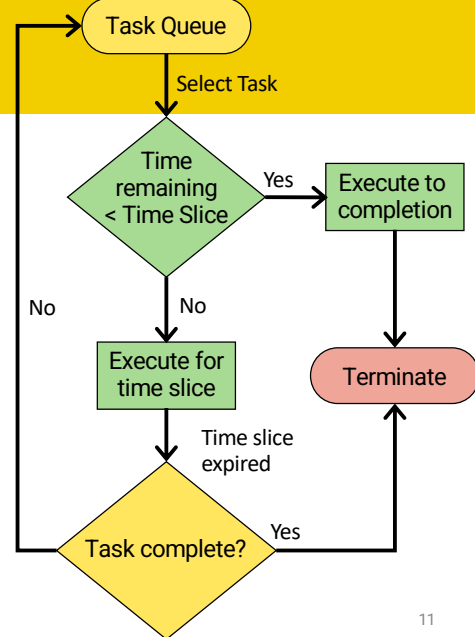
- Process ID, priority info, process state
- Program counter, stack pointer, general registers, others...
- Memory management info (main, shared data access)
- Scheduling info (timing requirements, time allocation)
- I/O status info (open files, resources)
- Processor status info
 - Clock frequency?
 - Sleep/Power mode?

Program as Collection of Tasks

- Partition overall program into tasks
- Task should do one thing, be as simple as possible
- Tasks can interact
 - Via synchronization primitives (e.g. semaphores), shared data structures, messages, etc
- When running, a task is assumed to have exclusive CPU access
 - Only one task is running at a time on the CPU
- As we have seen, tasks can be scheduled
 - Round-robin scheduler; fixed-priority, preemptive scheduler

Task Scheduling Events

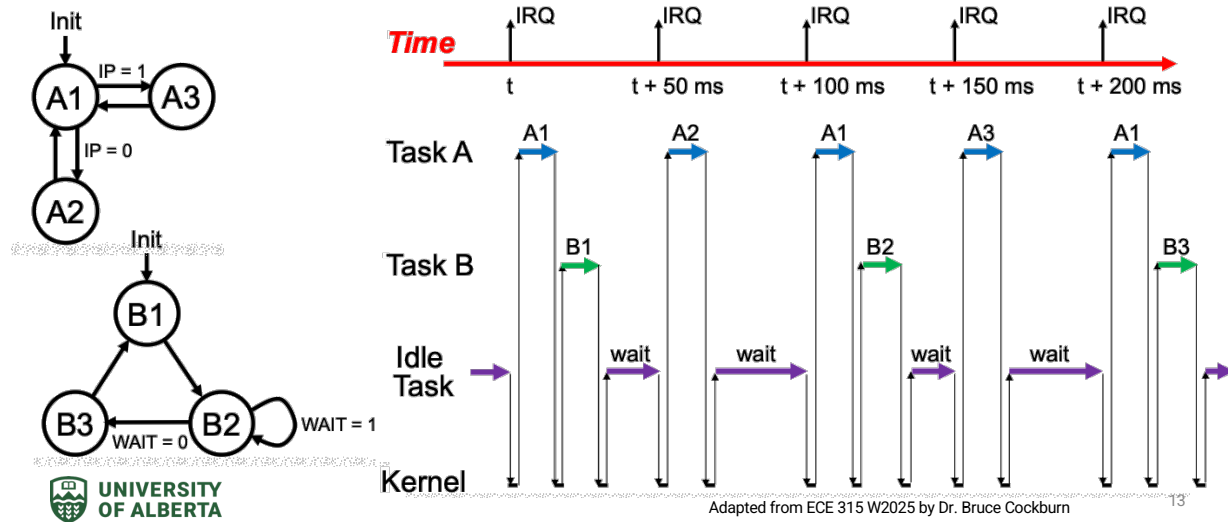
- The scheduler starts, stops, adds, removes, and suspends tasks
- The scheduler is invoked by a timer expiration interrupt
 - Current task is paused while scheduler runs
 - If task hasn't used all its time, the scheduler returns to the interrupted task
 - Otherwise, the task is "switched out" so the next task can run
 - Current task's state is saved in a TCB
 - New task's TCB is read to restore that task's state
 - New task is resumed



State-driven Code

- In some applications there is a natural mapping of tasks onto states
- Our example in the previous lecture makes use of states to change tasks in a fixed, timed loop.
- The tasks themselves can be state machines,
 - This is true of many other program types, from embedded to cloud
 - State machines are a very good way to organize event-oriented systems
- The key is not to get stuck in a state, either doing an indeterminate execution, or being blocked.

State-driven Code



The time in each state is not exactly harmonic, so the idle task pads things out.

The idle task must always run and can be used to do other things, like system housekeeping (servicing time tracking, resource usage, etc.)

State-driven Code

Advantages

- Many control problems (e.g., robotics), communications protocols, and signal processing problems involve distinct steps that must happen repeatedly in a specific sequence and at a specific frequency.
- The system architecture takes care of enforcing the state timing.
- Can easily have multiple state scheduling frequencies. A task could be allowed to change its scheduling frequency.

Disadvantages

- The task software must be partitioned into states that each execute relatively quickly. The partitioning of the software into states introduces structural complexity into the code.

Cooperative multitasking

- A task scheduling strategy where each task voluntarily yields control of the CPU to allow other tasks to run
- The scheduler does not preempt running tasks
 - each task must explicitly call a “yield” or “delay” function (or return to the scheduler) at appropriate times
- The system depends on the cooperation of tasks to share CPU time fairly and maintain system responsiveness
- When a task yields or delays, the next task in the round-robin queue is initiated
- There may still be interrupt for tasks to meet hard real-time requirements

Cooperative multitasking

Advantages

- Simplicity, easy to implement
- Minimal overhead compared to preemptive systems.
- Deterministic Behavior (if designed carefully)
- Few Context Switch Interruptions
- Low Resource Usage
- Avoids Race Conditions
 - Tasks don't overlap



Made with help from ChatGPT

Disadvantages

- Unresponsive if a Task misbehaves
- Manual Responsibility
 - yield points specific to task, which increases the risk of design errors
- Poor Real-Time Performance
- Difficult for Complex Systems
 - Doesn't scale well, becomes complicated
- Limited Interrupt Handling
 - task-level scheduling can't guarantee fast responses to events

16

Tasks vs. Processes vs. Threads

- In full-scale operating systems, like Unix, it is more common to use the terms “process” and “thread” instead of “task”
- A process (like a task) is associated with a program and a context; allocated memory regions, pointers, open files, sockets, etc.
- A thread (or light-weight process) is similar to a process, except that some resources may be shared with other active threads as part of a single process (e.g., files, memory space, memory pointers, input/output resources)
- Each thread has enough state information to be stopped
- and restarted independently (just like the TCB)

Real-Time Kernels

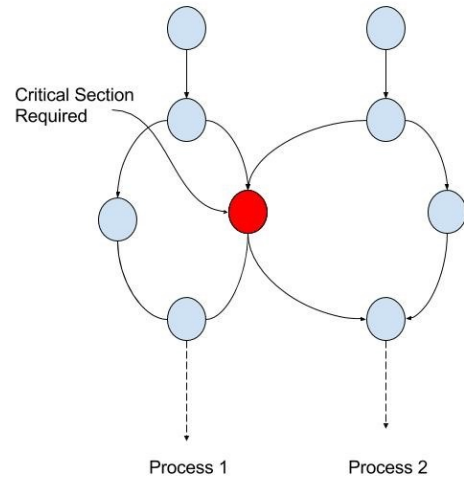
- A kernel provides the most basic services that would be expected in a multitasking operating system:
 - task/process creation, initiation, termination
 - task/process suspension and unsuspension
 - (possibly) timer-controlled time slicing
 - (possibly) process priorities and pre-emption rules
 - exception and interrupt handling mechanisms
 - basic inter-process communication services
 - mechanisms for protecting critical sections
 - inter-process synchronization features
- A small-scale system may have an operating system that is really just a kernel on its own.
- A real-time kernel is one that is intended to be used in systems that must meet hard real-time constraints.

Critical (Atomic) Sections

- A **critical section** — also called an **atomic section** — is a portion of code that must be executed completely without interruption because it accesses shared resources (such as global variables, hardware registers, or peripherals) that could be corrupted if another task or interrupt modifies them at the same time
- In simple terms, it's a protected block of code where the system ensures that only one piece of code accesses a shared resource at a time
- A key example is when a context switch is made
 - The TCB must be saved atomically, cannot be corrupted

Critical (Atomic) Sections

- Two processes may be executing in a multi-task pre-emptive system
- At a point, both need to access the same memory
- The time slices may be small enough that while Process 1 is in the midst of changing a variable, it is pre-empted and Process 2 changes something it shouldn't



Protecting critical sections – 4 strategies

- Disable all interrupts and disable task context switching
 - **Advantage:** Simple. Will work if new interrupt sources are added
 - **Disadvantage:** Too disruptive to the system? Might not be possible; some IRQs must be maintained (e.g., timer for the scheduler)
- Disable select interrupts and disable task context switching.
 - **Advantages:** Simple and less disruptive than disabling all IRQs
 - **Disadvantage:** Sometimes hard to determine those interrupts. The situation may change as new interrupts are introduced into a system that is modified and updated over time

Protecting critical sections – 4 strategies

- Disable select interrupts and multitasking, that is, disable task context switching.
 - **Advantage:** Simple
 - **Disadvantage:** Many tasks are disturbed, which is disruptive to system. Critical sections must still be protected from interrupts.
- Use semaphores or flags to protect critical sections.
 - **Advantage:** Minimizes the number of tasks that are affected.
 - **Disadvantage:** Critical sections within the semaphore functions must be protected from interrupts by disabling interrupts.

FreeRTOS Protection of Critical Sections;

`taskENTER_CRITICAL() ; taskEXIT_CRITICAL() ;`

1. Disable (mask) interrupts and suspend multitasking at start of critical section
2. Re-enable (unmask) interrupts and multitasking at end of section

```
taskENTER_CRITICAL(); /* nesting_level++; disable IRQs */
/* The critical section appears here. */
taskEXIT_CRITICAL(); /* nesting_level--;
if (nesting_level == 0) enable IRQs */
```

Advantages:

- Use a proven macro in high-level code.
- Fast to execute.
- Easy to include in C application code.

Disadvantages:

- Unnecessarily disruptive to the system?
- However, in some kernels (e.g., FreeRTOS), interrupts above some priority level can be left enabled and are not affected.



Adapted from ECE 315 W2025 by Dr. Bruce Cockburn

23

FreeRTOS Protection of Critical Sections; `vTaskSuspendAll()` ; `vTaskResumeAll()` ;

1. Suspend the scheduler and stop context switching. IRQs still enabled, but context switches they trigger are deferred until scheduler is re-enabled
2. Restore the scheduler and re-enable context switching

```
vTaskSuspendAll(); /* nesting_level++; disable scheduler */
/* The critical section appears here. */
xTaskResumeAll(); /* nesting_level--;
if (nesting_level == 0) enable scheduler
Returns pdTRUE if context switch occurred immediately. */
```

Advantages:

- Easy to include in C application code.

Disadvantages:

- Unnecessarily disruptive to the system?
- All tasks are affected, even those that will never attempt to access the critical section. Interrupts can still cause problems.



Adapted from ECE 315 W2025 by Dr. Bruce Cockburn

24

Protection of Critical Sections – Semaphores & Mutexes

- **Mutexes** and **semaphores** are *synchronization* primitives
 - are tools used in multitasking embedded RTOSes
 - coordinate access to shared resources and control task execution order
- A **semaphore** is a counter-based synchronization object that controls access to a shared resource by counting available slots or permissions.
- A **mutex** is like a binary semaphore but designed specifically for mutual exclusion – ensuring only one task at a time can access a shared resource (like a UART, sensor, or global variable)

Without synchronization, race conditions, data corruption, or deadlocks can occur.



Semaphore Types

Binary Semaphore

- Has only two states: taken (0) or available (1)
- Typically used for task synchronization (e.g., signaling from an ISR to a task)

Counting Semaphore

- Can hold values greater than 1
- Useful for resource pools (e.g., multiple identical hardware channels)

Mutex (Mutual Exclusion Object)

- A **mutex** (mutual exclusion lock) is a synchronization primitive that lets only one task/thread at a time enter a critical section that accesses shared resource (e.g., a peripheral, buffer, or global variable)
- All other tasks/threads either block or time out until the owner releases it.

Mutexes and semaphores ensure safe and predictable access by enforcing mutual exclusion or controlled signaling between tasks

Mutexes and Semaphores

Mutexes and semaphores ensure safe and predictable access by enforcing mutual exclusion or controlled signaling between tasks

- Key Differences Between Mutex and Binary Semaphore

Feature	Mutex	Binary Semaphore
Ownership	Has an owner task	None
Priority Inheritance	Yes (to prevent priority inversion	No
Typical Use	Protecting shared resources	Signaling between tasks, or between an ISR and task

Next Lecture

- Quiz #1
- Mutexes and Semaphore example
- Synchronization
- Priority Inversion
- Deadlock

Questions

